



[ECHO]

Rapport Final de Projet

Metroidvania avec écholocalisation

Projet informatique — Première année EPITA

Team Nightberry

Équipe projet

Amaury Giraud-Laforet

Eric Sahakian

Florian Croiset

Gaspard Sapin

Jules Cohen

Mai 2026

Table des matières

1	Introduction générale	5
1.1	Contexte et origine du projet	5
1.2	Présentation de l'équipe	6
1.3	Objectifs généraux	7
1.4	Plan global du rapport	7
2	Reprise du cahier des charges	8
2.1	Présentation du cahier des charges initial	8
2.1.1	Périmètre fonctionnel	8
2.1.2	Contraintes techniques	8
2.1.3	Éléments hors périmètre initial	9
2.2	Évolutions et écarts par rapport au CdST	9
2.3	Cahier des charges final	10
3	Organisation et gestion de projet	11
3.1	Répartition des tâches et évolution du leadership	11
3.2	Outils utilisés	11
3.3	Planning prévu et déroulement réel	12
3.4	Difficultés d'organisation rencontrées	13
3.5	Ce que l'organisation nous a appris	14
4	Réalisations individuelles	15
4.1	Amaury Giraud-Laforet	15
4.1.1	Périmètre de la mission	15
4.1.2	Évolution du rôle au fil des phases	16
4.1.3	Choix techniques	16
4.1.4	Réalisations concrètes	17
4.1.5	Joies et peines	20
4.1.6	Lien avec le travail des autres membres	21
4.2	Florian Croiset	22
4.2.1	Périmètre de la mission	22

4.2.2	Évolution du rôle au fil des phases	22
4.2.3	Choix techniques	23
4.2.4	Réalisations concrètes	24
4.2.5	Joies et peines	25
4.2.6	Lien avec le travail des autres membres	26
4.3	Eric Sahakian	27
4.3.1	Périmètre de la mission	27
4.3.2	Évolution du rôle au fil des phases	27
4.3.3	Choix techniques	28
4.3.4	Réalisations concrètes	29
4.3.5	Joies et peines	30
4.3.6	Lien avec le travail des autres membres	31
4.4	Gaspard Sapin	31
4.4.1	Périmètre de la mission	31
4.4.2	Évolution du rôle au fil des phases	32
4.4.3	Choix techniques	33
4.4.4	Réalisations concrètes	34
4.4.5	Joies et peines	35
4.4.6	Lien avec le travail des autres membres	36
4.5	Jules Cohen	36
4.5.1	Périmètre de la mission	36
4.5.2	Évolution du rôle au fil des phases	37
4.5.3	Choix techniques	37
4.5.4	Réalisations concrètes	38
4.5.5	Joies et peines	39
4.5.6	Lien avec le travail des autres membres	40
5	Récit de la réalisation collective	41
5.1	Vue d'ensemble du déroulement	41
5.1.1	Phase 1 : exploration et preuve de concept	41
5.1.2	Phase 2 : intégration et décisions difficiles	41
5.1.3	Phase 3 : aboutissement et finitions	42
5.2	Les joies collectives	42
5.3	Les peines collectives	42
5.4	Ce que le groupe a appris	43

5.5	Ce qui serait fait différemment	43
6	Présentation du résultat final	45
6.1	Description fonctionnelle du livrable	45
6.1.1	Gameplay central	45
6.1.2	Déplacements et capacités	45
6.1.3	Combat et progression	45
6.1.4	Système d'âmes	46
6.1.5	Multijoueur coopératif	46
6.1.6	Interface et accessibilité	46
6.2	Comparaison avec les objectifs initiaux	47
6.3	Démonstration écrite du déroulement d'une partie	48
6.4	Limites connues et pistes d'amélioration	48
7	Conclusion	50
7.1	Bilan synthétique du projet	50
7.2	Compétences développées	50
7.3	Ouvertures et perspectives	51
	Bibliographie et références	52
A	Architecture technique détaillée	53
A.1	Structure des modules	53
A.2	Protocole réseau	53
A.3	Système d'animation Aseprite	53
A.4	Système de cache et optimisations	54
B	Aperçus du jeu	55
B.1	Inspirations visuelles	55
B.2	Cartes du jeu	57
B.3	Sprites des personnages	59
B.4	Démonstration en jeu	59
C	Extraits de code commentés	60
C.1	Algorithme de patrouille réactive des ennemis	60
C.2	Algorithme DDA pour l'écholocalisation	62
C.3	Machine à états du boss	63

C.4	Protocole réseau	63
-----	----------------------------	----

Chapitre 1

Introduction générale

1.1 Contexte et origine du projet

Dans le cadre de notre première année à l'EPITA, il nous a été demandé de concevoir et développer un jeu vidéo complet en équipe, en Python et avec la bibliothèque Pygame. Ce projet, d'une durée d'environ huit mois (de septembre 2025 à mai 2026), constitue une mise en pratique des compétences acquises tout au long de l'année sur le travail en équipe et la conduite d'un projet d'envergure.

Notre équipe, la **Team Nightberry**, a choisi de créer **ECHO**, un jeu de plateforme 2D de type Metroidvania se déroulant dans un univers plongé dans l'obscurité totale. Le concept central repose sur l'écholocalisation : le joueur émet des ondes sonores qui révèlent progressivement et définitivement les contours du décor autour de lui. Cette mécanique originale, inspirée de jeux comme *Terraria* et *Dead Cells*, propose une expérience immersive où l'exploration et la maîtrise de l'environnement sont au cœur du gameplay. Les références visuelles qui ont guidé la direction artistique du projet sont présentées en annexe B ; on y retrouve notamment l'influence de *Terraria* pour le style pixel art contrasté et de *Silksong* pour la verticalité des environnements souterrains.

Le choix de ce concept n'a pas été immédiat. Lors des premières semaines du projet, plusieurs idées ont été explorées collectivement : un jeu de plateforme classique, un jeu de gestion, et même un jeu de type *roguelike* procédural. C'est finalement la piste de l'écholocalisation qui a fait l'unanimité, car elle répondait à la fois à une exigence d'originalité, à une faisabilité technique en Python et Pygame, et à une vision artistique cohérente. La référence à la biologie animale — les chauves-souris et les dauphins naviguent par écholocalisation — a rapidement enrichi le concept, jusqu'à devenir le fil directeur de l'ensemble de la direction artistique.

Le projet a également nécessité d'intégrer des composantes obligatoires

fixées par le règlement pédagogique : l'utilisation de PyGame, une forme d'intelligence artificielle, un site internet et un mode multijoueur en réseau. Ces contraintes ont influencé nos choix d'architecture dès les premières semaines et ont guidé une grande partie de nos décisions techniques tout au long de l'année. Loin de nous avoir freinés, ces exigences ont souvent constitué des leviers créatifs, nous poussant à trouver des solutions élégantes pour intégrer, par exemple, un réseau TCP robuste dans une boucle de jeu temps réel.

1.2 Présentation de l'équipe

Le projet [ECHO] a été réalisé par une équipe de cinq étudiants. Les responsabilités ont été réparties dès le début du projet, chaque membre se voyant confier un ou plusieurs domaines techniques, en tenant compte de ses compétences, de ses affinités, et de la charge de travail globale à distribuer équitablement.

Membre	Domaine de responsabilité principal
Amaury Giraud–Laforet	Chef de projet, Intelligence artificielle et objets
Florian Croiset	Menus, site web, musique et effets sonores
Eric Sahakian	Level design et graphismes (direction artistique)
Gaspard Sapin	Mécaniques de jeu, déplacements et boss
Jules Cohen	Réseau (multijoueur), lore et jouabilité

Cette répartition n'a pas été figée dès le premier jour. Elle a évolué au fil des semaines en fonction des besoins du projet, des imprévus techniques et de la disponibilité de chacun. Plusieurs membres ont ponctuellement apporté leur aide sur des domaines qui ne relevaient pas de leur responsabilité principale, notamment lors de périodes de surcharge ou de blocages. L'exemple le plus marquant est l'aide spontanée de Jules Cohen sur la correction des calques Tiled lors de la phase 2, qui a permis à Eric Sahakian de débloquer l'intégration de la map à un moment critique du projet.

Il convient également de noter le changement de chef de projet intervenu entre la phase 1 et la phase 2. Gaspard Sapin avait assuré ce rôle lors du

lancement du projet, pour ensuite le céder à Amaury Giraud-Laforet, afin que chacun puisse se concentrer pleinement sur son domaine de prédilection. Cette transition s’est faite naturellement, sans friction, et a contribué à stabiliser l’organisation de l’équipe pour la durée restante du projet.

1.3 Objectifs généraux

Le projet ECHO s’est fixé trois grands objectifs, en cohérence avec les exigences pédagogiques et notre vision du jeu :

- **Proposer une mécanique de jeu originale** : L’écholocalisation comme seul moyen de percevoir l’environnement, avec révélation progressive et permanente de la carte via un système de raycasting.
- **Livrer un jeu complet et jouable** : Une map entière intégrée avec ses textures, des personnages animés, des ennemis dotés d’une intelligence artificielle fonctionnelle, un boss, et un mode coopératif en réseau permettant à plusieurs joueurs de progresser ensemble.
- **Maîtriser les outils du développement en équipe** : Gestion de versions avec Git, organisation des tâches, communication régulière et création d’un livrable documenté et packagé.

1.4 Plan global du rapport

Ce rapport de projet final est structuré en six chapitres principaux.

Le chapitre 2 reprend et analyse le cahier des charges initial en le comparant à la réalité du projet livré. Le chapitre 3 décrit l’organisation de l’équipe, les outils utilisés et le déroulement du projet dans le temps. Le chapitre 4 présente en détail les réalisations de chaque membre. Le chapitre 5 explore l’aventure collective, avec ses réussites et ses difficultés. Le chapitre 6 présente le résultat final et le compare aux objectifs initiaux. Enfin, le chapitre 7 conclut le rapport par un bilan des compétences développées et des perspectives pour la suite.

Chapitre 2

Reprise du cahier des charges

2.1 Présentation du cahier des charges initial

2.1.1 Périmètre fonctionnel

Le cahier des spécifications techniques (CdST), validé en novembre 2025, définissait les fonctionnalités attendues autour de quatre axes :

- **Mécanique principale d'écholocalisation** : Le joueur émet une onde à 360° qui révèle les tuiles proches, avec une portée et un temps de recharge définis. Une variante directionnelle était prévue comme capacité déblocable.
- **Gameplay et progression Metroidvania** : Déplacements avancés (saut, double saut, dash), combats au corps à corps, ennemis avec IA, boss de fin de zone, et système d'objets collectables permettant de débloquent de nouvelles capacités.
- **Mode multijoueur coopératif en réseau** : Jusqu'à trois joueurs pouvant se connecter via TCP, avec synchronisation de l'état du jeu par un serveur autoritaire.
- **Ambiance et lore** : Direction artistique en pixel art, musique d'ambiance, effets sonores, et éléments de lore intégrés à l'environnement sous forme de pancartes interactives.

2.1.2 Contraintes techniques

Le projet devait respecter les contraintes suivantes :

- Développement en **Python 3** avec **Pygame**.
- Compatibilité avec **Windows 10/11**.
- Présence d'une **intelligence artificielle** et d'un **mode multijoueur en réseau**.
- Livraison d'un **exécutable** autonome, accompagné des sources, d'un site web et d'un manuel d'installation et d'utilisation.

2.1.3 Éléments hors périmètre initial

Certains éléments n'étaient pas explicitement prévus dans le CdST initial mais ont été ajoutés au fil du projet suite à des décisions d'équipe :

- Un système de relay TCP permettant de jouer en ligne sans configuration réseau, via un code de salle à six caractères.
- Un tutoriel interactif au premier lancement pour guider les nouveaux joueurs.
- Un système de journal de quêtes avec icône HUD et parchemin plein écran.
- Des effets de distorsion visuelle lors de l'activation de l'écho.

2.2 Évolutions et écarts par rapport au CdST

Au fil des trois phases de développement, plusieurs ajustements ont été apportés au cahier des charges initial. Certains sont liés à la découverte de contraintes techniques imprévues, d'autres à une réévaluation de nos ambitions en fonction du temps disponible.

Le principal écart concerne l'**intelligence artificielle**. Initialement prévue avec un algorithme A* complet dès la deuxième soutenance, son intégration a été reportée à la troisième phase en raison de la dépendance forte entre le pathfinding et la stabilité de la tilemap. Cette décision a permis de ne pas bloquer les autres parties du projet et de livrer une IA fonctionnelle dans le produit final, même si l'A* n'a pas pu être intégré au livrable faute de temps de test suffisant.

L'**animation des personnages** a également évolué : prévu pour les trois personnages jouables, le travail d'animation s'est avéré plus chronophage que prévu. Les animations complètes (déplacement, attaque, dégâts, mort) ont finalement été intégrées en phase 3, via le système Aseprite JSON, pour tous les sprites du jeu.

Le **multijoueur**, initialement prévu uniquement en réseau local (LAN), a été étendu avec un système de relay TCP permettant de jouer en ligne sans redirection de port. C'est une évolution positive non prévue, rendue possible par le bon avancement de la partie réseau en fin de phase 2.

2.3 Cahier des charges final

La version définitive du cahier des charges, telle qu'elle ressort des ajustements successifs, peut se résumer ainsi :

- **Mécanique d'écholocalisation** radiale avec révélation progressive et permanente de la carte.
- **Système de déplacement complet** : marche, saut, double saut, dash, avec capacités déblocables via des orbes sur la carte.
- **Combat et ennemis** avec trois types (patrouilleur, garde, gardien), système d'âmes perdues, boss Demon Slime.
- **Map complète** avec tileset graphique définitif, checkpoints, portes, clés et orbes de capacité.
- **Sprites et animations** pixel art pour les joueurs et le boss, réalisés avec Aseprite.
- **Intelligence artificielle** pour les ennemis, basée sur un système de patrouille réactif.
- **Multijoueur coopératif** jusqu'à 3 joueurs, via TCP avec serveur autoritaire, en LAN ou en ligne via code de salle.
- **Ambiance sonore complète** (musique, effets sonores, torche).
- **Menus, paramètres, sauvegarde**, tutoriel, site web et lore intégré.

L'architecture technique retenue pour répondre à ces exigences est décrite en détail dans l'annexe A, qui présente l'organisation des modules, le protocole réseau et les principaux mécanismes d'optimisation. Les extraits de code commentés figurant en annexe C illustrent concrètement les choix d'implémentation les plus structurants : l'algorithme de patrouille des ennemis, le raycasting DDA et le protocole TCP à préfixe de longueur.

Chapitre 3

Organisation et gestion de projet

3.1 Répartition des tâches et évolution du leadership

Dès la constitution de l'équipe, Gaspard Sapin a été désigné chef de projet pour la phase initiale. Il a ensuite cédé sa place à Amaury Giraud-Laforet, qui a repris ce rôle pour la suite du projet. Ce changement a permis à Gaspard de se concentrer pleinement sur les mécaniques de jeu, qui constituaient son domaine de prédilection et une composante critique du gameplay.

La répartition des tâches a été établie en tenant compte des affinités et des compétences de chacun, tout en assurant une couverture de toutes les composantes du projet. Elle a été formalisée dans un tableau Notion dès les premières semaines, et révisée à chaque début de phase pour s'adapter aux évolutions du projet.

3.2 Outils utilisés

Choix techniques

- **Gestion de versions** : Git et GitHub, avec un dépôt centralisé et des branches par fonctionnalité. Les pull requests étaient relues par au moins un autre membre avant d'être fusionnées.
- **Communication** : Discord, avec un serveur dédié comportant des canaux par domaine (code, graphisme, réseau, sons) et des réunions vocales hebdomadaires.
- **Suivi de tâches** : Notion, avec un tableau Kanban et des colonnes À faire / En cours / Terminé, complété par des fiches de fonctionnalité détaillant les critères d'acceptation.
- **Graphisme** : Aseprite, pour la création des sprites et animations en

- pixel art, avec export JSON pour l'intégration dans Pygame.
- **Level design** : Tiled Map Editor, pour la conception de la carte sous forme de fichier TMX exploitable directement par le moteur.
 - **Packaging** : PyInstaller, pour la génération de l'exécutable Windows autonome.
 - **Site web** : Hébergé sur GitHub Pages, avec Supabase pour la partie dynamique (base de données, votes, bot Discord).

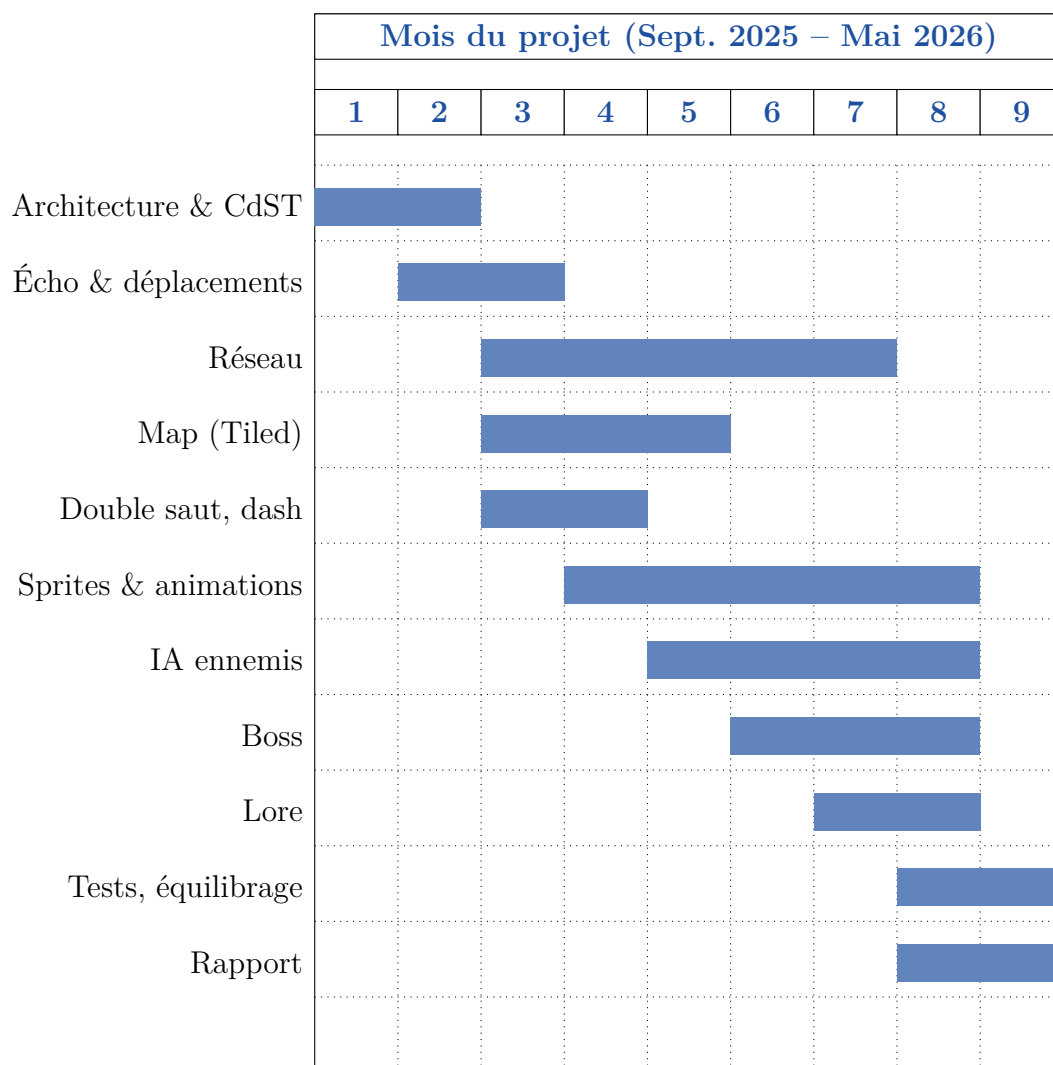
L'organisation modulaire du code qui a découlé de ces choix d'outils est documentée dans l'annexe A (section A.1), qui recense l'ensemble des répertoires du projet et la responsabilité de chacun. Cette structuration, établie dès la phase 1 et maintenue jusqu'à la livraison finale, a constitué un cadre stable dans lequel chaque membre pouvait travailler sans empiéter sur le domaine des autres.

3.3 Planning prévu et déroulement réel

Le projet s'est déroulé sur trois phases distinctes correspondant aux trois soutenances. Le tableau suivant résume les objectifs de chaque phase et leur réalisation effective :

Phase	Période	Objectifs principaux
Phase 1 (S1)	Sept.–Janv. 2026	Architecture, écho, mouvements de base, PoC réseau
Phase 2 (S2)	Janv.–Mars 2026	Double saut, dash, map partielle, multijoueur LAN
Phase 3 (S3)	Mars–Mai 2026	Map complète, sprites animés, IA ennemis, boss, lore

Le diagramme de Gantt ci-dessous illustre le déroulement effectif des principales tâches du projet :



Les principaux glissements constatés concernent l'intégration de l'IA (repoussée de la phase 2 à la phase 3) et les animations des personnages (plus longues à produire que prévu). Ces deux points sont détaillés dans les sections individuelles correspondantes.

3.4 Difficultés d'organisation rencontrées

La principale difficulté organisationnelle a été la gestion des dépendances entre modules. Plusieurs composantes du jeu étaient fortement couplées : la map devait être stable avant d'intégrer l'IA, les animations devaient être finalisées avant de les charger en code. Ces contraintes ont parfois créé des situations de blocage en cascade, où l'avancement d'un membre dépendait entièrement de la livraison d'un autre.

La fusion des branches Git a également posé des difficultés ponctuelles,

notamment lors de l'intégration simultanée du système réseau et des mécaniques de jeu en phase 2. Plusieurs sessions de résolution de conflits ont été nécessaires, parfois longues et délicates.

Enfin, la disponibilité variable des membres selon les périodes de partiels ou de rendu d'autres projets a nécessité une certaine souplesse dans le planning, avec des redistributions de charge ponctuelles entre les membres.

3.5 Ce que l'organisation nous a appris

Cette expérience de gestion de projet sur huit mois nous a appris plusieurs leçons concrètes. La première est l'importance d'identifier les dépendances critiques en amont : savoir qu'une fonctionnalité bloquera les autres si elle n'est pas livrée à temps permet de la prioriser correctement. La deuxième est la valeur d'une communication régulière et transparente sur l'état d'avancement, même et surtout quand il y a des retards. La troisième est que les outils de collaboration (Git, Notion, Discord) ne remplacent pas les échanges humains, mais les rendent plus efficaces.

Chapitre 4

Réalisations individuelles

Le cœur de ce rapport est constitué des contributions individuelles de chaque membre. Chaque section détaille les objectifs assignés, les choix réalisés, les réalisations concrètes, les joies et les peines, ainsi que les liens avec le travail des autres membres.

4.1 Amaury Giraud–Laforet

Chef de projet — Intelligence artificielle — Objets

4.1.1 Périmètre de la mission

En tant que chef de projet, j'avais la responsabilité globale de la coordination de l'équipe et de la cohérence technique du projet. Sur le plan technique, mes domaines principaux étaient l'intelligence artificielle des ennemis et du boss, l'implémentation des objets interactifs du monde (âmes, orbes, clé, porte, torche, pancartes), ainsi que la supervision artistique des éléments graphiques produits par Eric.

Ce périmètre large, à cheval entre les responsabilités de gestion et les contributions techniques, a rythmé l'ensemble des huit mois de développement. Chaque semaine, je devais arbitrer entre avancer sur mon propre code et répondre aux besoins de coordination de l'équipe. Cette tension permanente entre contribution individuelle et rôle transversal est probablement l'aspect le plus formateur de l'ensemble de cette expérience.

Ma mission de chef de projet comprenait également la tenue du tableau Notion, l'animation des réunions hebdomadaires sur Discord, la gestion des conflits de priorités entre membres, et la rédaction des bilans d'avancement avant chaque soutenance. Ces responsabilités invisibles dans le livrable final ont pourtant occupé une part significative de mon temps tout au long du projet.

4.1.2 Évolution du rôle au fil des phases

Lors de la première soutenance, l'objectif sur la partie IA était de poser les bases conceptuelles : choix de l'algorithme, définition de l'architecture, et premiers tests locaux. La partie objets devait atteindre environ 20 % d'avancement, avec la création d'une classe de base et les premières réflexions sur l'architecture orientée objet.

En phase 1, l'essentiel de mon travail technique a porté sur la conception architecturale plutôt que sur l'implémentation directe. J'ai défini les interfaces entre les modules (entités, serveur, carte) et posé les bases de la classe d'objets générique. Pendant cette phase, l'IA n'a pas encore été codée dans le dépôt commun : les recherches théoriques sur l'algorithme A* et les machines à états finis ont été menées, des vidéos et articles de référence consultés (notamment Red Blob Games pour A*), mais l'implémentation a été délibérément différée, en attente d'une tilemap stable.

En phase 2, la classe `AmePerdu` a été concrètement mise en jeu, le premier objet à états (la torche) a été implémenté, et les premières expérimentations de pathfinding ont été menées en local sans être fusionnées dans le dépôt. C'est aussi au cours de cette phase que j'ai pris la mesure de la dépendance critique entre l'IA et la stabilité de la map : chaque modification de la tilemap rendait caducs les tests précédents de l'algorithme A*.

En phase 3, l'ensemble des objets interactifs a été finalisé et intégré, le système d'IA a été complété avec la patrouille réactive, et l'implémentation partielle de l'A* a été poussée jusqu'à un état fonctionnel sur une map de test, même si elle n'a finalement pas rejoint le livrable final.

4.1.3 Choix techniques

Choix techniques

- **Abandon de l'A* au profit d'un algorithme de patrouille réactif** : Initialement prévu avec un pathfinding A* complet dès la deuxième soutenance, l'IA a dû être repensée en cours de projet. Le retard lié à la dépendance vis-à-vis de la tilemap a conduit à opter pour un algorithme de patrouille déterministe enrichi de règles

comportementales. Ce choix pragmatique a permis de livrer une IA fonctionnelle et testable sans bloquer le reste du développement.

- **Typage d'ennemis par configuration** : Plutôt que de créer une classe par type d'ennemi, un système de configuration par paramètres (vitesse, PV, taille, argent lâché, préfixe d'asset) permet de définir trois archétypes distincts à partir d'une classe de base commune. Cette approche data-driven simplifie l'ajout de nouveaux types d'ennemis et rend l'équilibrage indépendant du code.
- **Architecture objet unifiée pour les items** : Les objets du jeu partagent une interface commune (`get_etat` / `set_etat`) pour la sérialisation réseau, garantissant une intégration homogène dans le serveur autoritaire. Cette décision, prise dès la phase 1, a considérablement simplifié le travail de Jules lors de l'intégration réseau des objets.
- **Aseprite JSON pour les animations** : Les sprites animés sont exportés depuis Aseprite au format JSON+PNG. La classe `BossAnimator` lit ce format et découpe les frames par tag, permettant une gestion propre des animations sans hardcoder les index dans le code.
- **Grille spatiale pour les collisions** : Une grille spatiale précalculée à partir de la tilemap permet de ne tester les collisions qu'avec les tuiles proches de l'entité, évitant une itération coûteuse sur l'intégralité de la carte à chaque frame. Ce choix d'optimisation, inspiré des techniques classiques de partitionnement spatial, était indispensable pour maintenir les performances avec plusieurs entités actives simultanément.

4.1.4 Réalisations concrètes

Intelligence artificielle : du retard de l'A à la patrouille réactive***Réalisations concrètes**

L'implémentation de l'IA des ennemis a connu un parcours atypique. Le cahier des charges initial prévoyait un pathfinding A* complet dès la deuxième soutenance. Cependant, l'algorithme A* sur une tilemap repose entièrement sur la stabilité de celle-ci : chaque nœud du graphe de navigation correspond à une tuile, et la moindre modification invalide le graphe. Or, la map définitive d'Eric n'a été stabilisée qu'en fin de phase 2, après de nombreuses corrections de calques et de collisions.

La décision collective a donc été de reporter l'intégration de l'A* à la phase 3 et d'utiliser en attendant un algorithme de remplacement plus robuste aux changements de la carte. L'algorithme finalement intégré est un système de **patrouille réactive à base de règles** : chaque ennemi avance dans une direction courante, et à chaque frame le moteur évalue deux conditions (présence d'un mur, absence de sol dans la direction du déplacement). Ce mécanisme garantit que les ennemis ne tombent jamais dans le vide et ne restent jamais bloqués dans un coin. La détection du joueur repose sur un rayon de vision circulaire configurable par archétype. Ce système a été enrichi de plusieurs comportements supplémentaires : un timer de flash sonar qui révèle brièvement les ennemis lors de l'écho du joueur, un cooldown d'attaque indépendant par ennemi avec des fenêtres de frames actives précises, et un timer de respawn configurable. Ces ajouts ont été réalisés de manière incrémentale, en testant chaque comportement séparément avant de les combiner.

Une implémentation partielle de l'A* existe dans le dépôt (`core/astar.py`), avec son service asynchrone associé (`core/pathfinding.py`) utilisant un threadpool pour ne pas bloquer la boucle de jeu. Cette implémentation est fonctionnelle sur une map de test minimaliste, mais n'a pas été intégrée au livrable final faute de temps de calibrage sur la map complète. C'est sans doute la principale dette technique du projet, et la première fonctionnalité qui serait intégrée si le développement devait se poursuivre. Le code complet de l'algorithme de patrouille réactive, avec ses règles comportementales et sa gestion des cooldowns, est reproduit et commenté en annexe C

(section C.1) ; il illustre concrètement le pragmatisme qui a guidé ce choix d'implémentation.

Système d'objets : trois catégories complémentaires

Réalisations concrètes

Les objets du monde d'[ECHO] ont été organisés en trois catégories fonctionnelles, dont l'architecture a été posée dès la phase 1 et enrichie progressivement jusqu'à la phase 3.

Objets fixes (décor interactif et informatif) :

Les pancartes de lore (**PancarteLore**) sont positionnées à des emplacements clés de la map. Elles sont verrouillées par défaut et débloquables contre 30 âmes, ce qui déclenche une popup de confirmation côté client et une transaction côté serveur. Une fois débloquées, elles affichent leur texte de lore dans une bulle de dialogue stylisée parchemin. Ce mécanisme de coût en âmes a été choisi pour créer un arbitrage intéressant pour le joueur : dépenser ses ressources pour de la narration, ou les conserver pour les achats de potions.

Objets ramassables (collecte et progression) :

Les âmes libres (**AmeLibre**) sont des cristaux flottants disséminés sur la carte, valant chacun 5 âmes. Elles constituent la récompense principale de l'exploration. Les âmes loot (**AmeLoot**) sont lâchées à la mort des ennemis avec une physique de dispersion : elles jaillissent dans plusieurs directions, rebondissent sur les murs et le sol avec amortissement, puis disparaissent après 30 secondes si elles ne sont pas ramassées. Ce comportement physique a été particulièrement soigné car il constitue un moment de plaisir immédiat après un combat. Les orbes de capacité (**OrbeCapacite**) débloquent définitivement une capacité du joueur au contact, et leur état est persisté dans la sauvegarde. La clé (**Cle**) fonctionne sur le même principe ; elle a été le premier objet ramassable implémenté, dès la phase 2, servant de prototype pour valider l'architecture du système d'inventaire.

Objets à phases (machine à états et déclencheurs) :

La porte (**Porte**, 64×96 px) possède trois états (fermée, en_ouverture, ouverte), une animation de transition et un son synchronisé. La torche (**Torche**) est un objet interactif à deux états dont l'activation génère un

halo lumineux de 220 px de rayon. Elle a été le premier objet à états entièrement fonctionnel du jeu, réalisé en phase 2, et a servi de gabarit technique pour la conception de la porte. L'âme perdue (*AmePerdue*), créée à la mort du joueur avec son argent accumulé, constitue le pilier de la mécanique inspirée de *Dark Souls* : la possibilité de récupérer sa mise après une mort risquée est l'un des ressorts de tension du gameplay.

Rôle de chef de projet : coordination et arbitrages

Au-delà du code, le rôle de chef de projet a représenté une charge de travail constante tout au long des huit mois. J'ai animé les réunions hebdomadaires sur le serveur Discord de l'équipe, rédigé les comptes-rendus, maintenu le tableau Notion à jour, et assuré le suivi des tâches critiques. Ce rôle m'a également conduit à prendre des décisions délicates, notamment celle de reporter officiellement l'A* en phase 2 après avoir constaté que son développement risquait de bloquer d'autres parties du jeu.

La gestion des dépendances entre modules a été l'aspect le plus complexe de ce rôle. J'ai dû cartographier les liens entre les différentes parties du code — la map bloque l'IA, les animations bloquent le chargement des sprites, le serveur dépend des interfaces des entités — et m'assurer que les livraisons se faisaient dans le bon ordre. Cette coordination implicite, souvent invisible, a été déterminante pour que le jeu soit jouable à chaque soutenance.

4.1.5 Joies et peines

Joies

- Voir les âmes loot fonctionner pour la première fois en vrai. Un ennemi abattu a projeté des orbes dans toutes les directions, ils ont rebondi sur le sol et les murs, puis se sont immobilisés un par un. C'est un de ces moments où on réalise que le jeu prend vraiment vie, où la mécanique qu'on avait imaginée sur le papier devient une expérience concrète et satisfaisante.
- La satisfaction d'avoir une architecture objet suffisamment propre

pour intégrer de nouveaux items rapidement. Les pancartes de lore, les potions et les âmes loot ont chacune été ajoutées en moins d'une journée grâce aux interfaces établies en début de projet. Ce gain de vitesse en fin de projet a validé les choix d'architecture faits bien plus tôt.

- Coordonner l'équipe sur plusieurs mois et voir le jeu prendre forme progressivement, avec les contributions de chacun qui s'assemblent. La dernière session de test avant rendu, avec les cinq membres connectés, a été un moment de satisfaction collective difficile à décrire.

- Le retard pris sur l'A* reste ma principale frustration. Avoir une implémentation fonctionnelle dans `core/astar.py` sans pouvoir l'intégrer au livrable final faute de temps de test a été difficile à accepter. Ce fichier dans le dépôt restera longtemps comme le symbole de quelque chose de presque accompli.
- La gestion simultanée du rôle de chef de projet et des responsabilités techniques a parfois été difficile. Il était compliqué de se concentrer profondément sur un problème de code tout en gérant les blocages des autres membres. Plusieurs fois, j'ai dû interrompre un débogage en cours pour répondre à une question ou arbitrer un conflit de priorités.
- La synchronisation des objets ramassables en multijoueur a posé des problèmes imprévus : deux joueurs pouvaient ramasser la même orbe simultanément si leurs paquets arrivaient dans le même tick serveur. Corriger ce bug a nécessité un travail non prévu initialement, et m'a appris que la concurrence est un problème difficile même à petite échelle.

4.1.6 Lien avec le travail des autres membres

Les ennemis que j'ai développés s'intègrent directement dans la boucle de jeu du serveur de Jules, qui appelle `appliquer_logique` à chaque tick. Les objets interactifs sont gérés par le serveur et synchronisés vers tous les clients via le protocole réseau. Les sprites créés en collaboration avec Eric sont chargés par la classe `Joueur` de Gaspard, et le système d'animations du boss alimente

sa FSM du `DemonSlimeBoss`. Le système de lore s'appuie sur les menus et le HUD de Florian pour son rendu côté client. Cette imbrication profonde de mes contributions dans toutes les autres parties du projet reflète bien la position transversale qu'occupait mon rôle de chef de projet.

4.2 Florian Croiset

Menus et interface utilisateur — Site web — Audio

4.2.1 Périmètre de la mission

Dans le projet Echo, je me suis occupé de trois domaines : le site web du projet, les menus et le système audio du jeu. Le site web devait servir à la fois de vitrine publique pour le jeu et d'outil interne à l'équipe, avec une page recensant tous les designs et assets du projet. Les menus représentaient la couche d'interface principale du jeu, de l'écran titre jusqu'aux paramètres et au tutoriel. Enfin, le système audio couvrait la musique de fond, les effets sonores et un système de son spatial pour certains éléments du jeu.

Ce périmètre, bien que varié, présente une cohérence forte : il s'agissait dans les trois cas de construire ce que le joueur voit et entend en dehors du gameplay pur. Ces éléments ont un impact décisif sur la première impression et sur l'identité globale du jeu, même s'ils sont souvent moins visibles que les mécaniques de jeu dans les démonstrations.

4.2.2 Évolution du rôle au fil des phases

Lors de la première soutenance, le site web était prévu à 30 % d'avancement, les menus devaient être complets à 100 % (objectif atteint dès la phase 1), et aucun travail musical n'était encore prévu.

Le site web a largement dépassé les attentes en phase 1, atteignant 70 % d'avancement alors que 30 % étaient prévus. Cette avance a permis d'utiliser le site activement tout au long du projet comme outil de communication interne et de partage des assets. Les versions 2.0 à 2.4 ont été publiées avant la soutenance 1, introduisant un backend Supabase, un dashboard administrateur, un bot Discord et une roadmap interactive.

En phase 2, le travail s'est concentré sur la sécurisation du site (version 2.5, audit complet des fichiers JavaScript et HTML, corrections de vulnérabilités). Du côté du jeu, la gestion musicale a été intégrée et le menu a été enrichi avec l'écran de connexion réseau et les options de reconfiguration complète des touches.

En phase 3, les effets sonores ont été ajoutés, le HUD a été enrichi d'un indicateur d'écho animé et d'une barre de vie pour le boss, et le tutoriel interactif a été développé.

4.2.3 Choix techniques

Choix techniques

- **HTML/CSS/JS avec Supabase et GitHub Pages pour le site** : Partir d'un stack web classique sans framework permettait un contrôle total sur le rendu et évitait d'introduire une complexité inutile pour un projet de cette échelle. Supabase a été choisi pour sa simplicité de mise en place et ses fonctionnalités intégrées (base de données, authentification, webhooks). GitHub Pages permettait un hébergement gratuit directement lié au dépôt, avec un déploiement automatique à chaque push sur la branche principale.
- **Module audio singleton** : Toutes les fonctions du module `music.py` opèrent sur un état global interne, sans instanciation. Ce choix simplifie l'accès depuis n'importe où dans le code sans avoir à propager une référence à travers toute l'arborescence. Il présente cependant un couplage fort qu'il faudrait reconsidérer dans une refonte future.
- **Charte graphique unifiée** : Toutes les constantes visuelles (couleurs, effets néon, animations de fond) sont centralisées dans `parametres.py` et réutilisées via des fonctions communes dans `ui/effets_visuels.py`, garantissant une cohérence entre tous les écrans. Ce choix a grandement facilité les révisions visuelles en cours de projet : modifier une couleur ou un effet se répercute automatiquement sur l'ensemble de l'interface.
- **Composants réutilisables** : La classe `Bouton` et le `Slider` ont été conçus comme des composants indépendants avec leurs propres états visuels (normal, survolé, pressé, désactivé), utilisables dans n'importe

quel menu sans duplication de code. Cette approche orientée composants, inspirée des frameworks web modernes, a été appliquée à un contexte Pygame en pur Python.

- **Architecture par états pour les menus :** L'ensemble du système de menus repose sur une variable `etat_jeu` qui détermine quel écran est affiché. Chaque état correspond à un module distinct avec sa propre logique de rendu et d'événements. Cette séparation claire a évité les enchevêtrements de code qui auraient rendu difficile l'ajout de nouveaux écrans en cours de projet.

4.2.4 Réalisations concrètes

Réalisations concrètes

Menus et interface :

Les menus couvrent l'intégralité du parcours utilisateur : écran de démarrage (splash screen avec fondu), menu principal avec titre en néon animé, menu de sélection de slots de sauvegarde (3 emplacements), menu de connexion réseau (IP directe ou code de salle), menu des paramètres (résolution, plein écran, reconfiguration des touches, volume, langue FR/EN), menu de pause et menu de confirmation. Chaque bouton utilise la classe réutilisable `Bouton` avec quatre styles visuels (normal, danger, confirm, ghost) et un effet de lueur néon au survol. Le menu principal a été fonctionnel dès la première soutenance, à 95 % de son état final, et constitue l'un des éléments les plus stables de tout le projet.

HUD en jeu :

Le HUD affiche en temps réel les PV du joueur sous forme de cœurs, l'argent (âmes), l'indicateur de cooldown de l'écho (arc circulaire animé avec décompte passant du rouge au cyan), les capacités débloquées, et la barre de vie du boss. Cet indicateur de cooldown est l'une des réalisations dont je suis le plus fier : il est à la fois informatif, esthétique et cohérent avec la charte néon du jeu. La barre de vie du boss, ajoutée en phase 3, synchronise son affichage avec la FSM du boss développée par Gaspard.

Tutoriel interactif :

Un tutoriel a été ajouté au premier lancement du jeu, guidant le joueur

à travers les contrôles de base et la mécanique d'écho. Ce tutoriel s'intègre naturellement dans les premiers instants de jeu sans interrompre l'exploration ; il repose sur des déclencheurs positionnels qui affichent des indications contextuelles au moment opportun.

Système audio :

Un module singleton `music.py` centralise l'initialisation du mixer, la gestion de la musique de fond (avec pause/reprise sur le menu), et la lecture de plus de quinze effets sonores distincts. La musique de fond, générée par intelligence artificielle en phase 2 puis affinée en phase 3, a été choisie pour son ambiance souterraine et oppressante, cohérente avec l'univers du jeu. Le volume de la torche est spatialisé dynamiquement selon la distance joueur-torche, une fonctionnalité discrète mais qui contribue significativement à l'immersion.

Site web :

Le site officiel propose une présentation du projet, une page design avec les assets du jeu, une roadmap interactive avec système de vote, un dashboard administrateur, un historique des versions et un bot Discord. Développé entièrement en HTML5/CSS3/JavaScript natif sans framework, il reflète un choix délibéré de maîtriser chaque ligne de code plutôt que de s'appuyer sur des abstractions. Il est accessible à l'adresse <https://florian-croiset.github.io/jeusite/>.

4.2.5 Joies et peines

Joies

- L'indicateur de cooldown de l'écho (arc circulaire animé passant du rouge au cyan) est l'une des réalisations dont je suis le plus fier. Il est à la fois informatif, beau et cohérent avec la charte graphique. Sa conception a demandé plusieurs itérations sur le calcul de l'arc et le choix de la palette de couleurs, mais le résultat final justifie pleinement cet investissement.
- Le site web, parti de zéro, a atteint un niveau de finition très satisfaisant avec des fonctionnalités dynamiques réelles, et l'équipe l'a utilisé activement tout au long du projet pour partager les avancées. Voir

mes camarades consulter la roadmap et voter pour les fonctionnalités a été une récompense inattendue.

- La réalisation du tutoriel en phase 3 a été l'une de mes contributions les plus valorisantes, car elle répond directement à une préoccupation d'accessibilité : un jeu dans le noir complet peut déstabiliser au premier lancement, et le tutoriel atténue cet effet de manière élégante.

- La cohérence visuelle des menus sur des résolutions d'écran très différentes a demandé un calibrage itératif long. Un bouton parfaitement proportionné sur mon écran pouvait sembler écrasé ou mal centré sur la machine de quelqu'un d'autre. L'absence d'un framework de mise en page adaptative s'est fait sentir ici.
- Certains effets sonores devaient être joués côté client et non côté serveur, ce qui a nécessité un refactoring du système d'événements sonores en cours de projet. Cette distinction client/serveur, évidente en théorie, s'est révélée délicate à implémenter dans une architecture où le serveur possède toute la logique de jeu.
- La création musicale par IA, si elle a permis d'obtenir rapidement un résultat acceptable, a laissé une légère frustration : la musique finale, bien qu'adaptée à l'ambiance, ne correspond pas exactement à la vision sonore que nous avons initialement imaginée pour Echo.

4.2.6 Lien avec le travail des autres membres

Les menus constituent l'enveloppe de tout le projet : le joueur passe par eux pour accéder au jeu de Gaspard et Jules, configurer les paramètres sauvegardés, ou rejoindre une partie réseau. Le HUD interagit directement avec l'état du joueur (Gaspard) et les données du serveur (Jules). Le système audio accompagne l'ensemble du jeu, des menus jusqu'aux combats, en s'appuyant sur les événements émis par les entités développées par Amaury et Gaspard. Le site web a servi de tableau de bord collectif pendant toute la durée du projet, centralisant les assets d'Eric et les annonces d'avancement de toute l'équipe.

4.3 Eric Sahakian

Level design — Direction artistique

4.3.1 Périmètre de la mission

Dans le projet Echo, je me suis occupé de deux domaines : le level design et le graphisme. Pour le level design, il s'agissait de concevoir la structure de la map, de définir la logique de progression propre au genre metroidvania, puis de l'intégrer dans le moteur. Pour le graphisme, le travail portait sur la direction artistique, la création des sprites et animations des personnages et ennemis, ainsi que la conception de quelques objets.

Ces deux domaines sont naturellement liés : la map est le terrain sur lequel tout le reste prend vie, et les choix graphiques ont régulièrement dû tenir compte de ce qui était techniquement faisable dans le moteur. Travailler simultanément sur le contenant (la carte) et sur les éléments visuels qui le peuplent a imposé une discipline de priorisation constante.

4.3.2 Évolution du rôle au fil des phases

Lors de la première soutenance, le level design devait atteindre 60 % d'avancement (structure définie, non encore intégrée dans le moteur) et le graphisme 20 % (direction artistique et premiers designs statiques, sans animations).

En phase 1, la map a été conçue en deux itérations sur papier, puis reproduite dans Tiled. L'avancement réel du level design a atteint 70 %, dépassant les prévisions. Pour le graphisme, la direction artistique et les designs des personnages principaux ont été réalisés avec Aseprite, atteignant 35 % — également au-delà des objectifs. Cependant, aucune animation n'avait encore été produite.

En phase 2, la carte a été intégrée dans Pygame, mais l'intégration s'est heurtée à des problèmes de calques qui ont mobilisé une part importante du temps disponible. Jules a proposé son aide à un moment critique, permettant de débloquer la situation. La moitié de la map est devenue jouable. En parallèle, les animations de base (course, saut) ont été produites pour un des personnages,

sans encore être intégrées dans le code.

En phase 3, l'intégralité de la map a été ouverte au joueur, les textures définitives appliquées, et toutes les animations complètes (idle, course, saut, attaque, dégâts, mort) intégrées pour deux personnages jouables et quatre ennemis.

4.3.3 Choix techniques

- **Tiled pour la map** : Bien adapté aux tilemaps pixel art, il permet de travailler avec des calques distincts pour séparer décor, collisions et éléments interactifs, ce qui est indispensable pour que le moteur sache sur quoi le joueur peut marcher. La distinction entre calque de collision et calque décoratif est au cœur de l'architecture de la carte.
- **Aseprite pour les graphismes** : Outil spécialisé pixel art, très pratique pour créer des animations frame-by-frame. L'interface est pensée pour ça, avec une gestion native des tags d'animation et un export JSON permettant une intégration directe dans le moteur via la classe `BossAnimator`. Le workflow Aseprite → JSON → Pygame s'est révélé particulièrement efficace une fois maîtrisé.
- **Pixel art volontairement lisible** : Pygame impose des contraintes de performance, et un sprite trop détaillé aurait aussi rendu l'animation beaucoup plus longue à produire. Rester simple était autant un choix technique que pratique. La lisibilité des sprites dans un environnement sombre a guidé le choix des contrastes et des palettes de couleurs.
- **Conception itérative (papier → Tiled → Pygame)** : Commencer sur papier permet de détecter les problèmes de progression tôt, avant de se retrouver à tout retravailler dans l'outil. Les deux itérations papier réalisées en phase 1 ont révélé plusieurs incohérences dans la logique de progression qui auraient été coûteuses à corriger directement dans Tiled.

4.3.4 Réalisations concrètes

Réalisations concrètes

Level design :

La map a commencé sur papier, en deux versions successives, pour poser la structure générale et réfléchir aux zones bloquées et à ce qui permet de les débloquent. La première version, trop linéaire, a été abandonnée au profit d'une seconde version exploitant davantage la verticalité et les embranchements. Cette approche itérative, avant même d'ouvrir Tiled, a permis de détecter les problèmes de progression tôt et d'éviter des refactorings coûteux.

La map finale prend la forme d'un réseau de salles et tunnels interconnectés, avec de nombreuses bifurcations et quelques zones secondaires pour encourager l'exploration. La progression fonctionne comme dans tout metroidvania : le double saut ouvre des zones en hauteur, certaines portes nécessitent des objets spécifiques, et les boss débloquent de nouvelles capacités. La conception a été pensée pour que le joueur revienne naturellement sur ses pas, découvrant des zones auparavant inaccessibles à mesure qu'il acquiert de nouvelles capacités.

L'intégration dans Pygame a été la partie la plus difficile. Le problème principal venait des calques : des éléments comme le sol s'étaient retrouvés sur un calque décoratif, ignoré par le système de collisions, ce qui provoquait des murs invisibles et des chutes dans le vide. Jules a proposé de m'aider sur Tiled à un moment critique, ce qui m'a vraiment soulagé et a permis d'avancer bien plus vite. Ce travail collaboratif sur les calques a abouti à une convention documentée : calque 0 pour la décoration de fond, calque 1 pour les collisions, calque 2 pour les éléments interactifs.

Graphisme :

La direction artistique s'inspire de jeux comme Dead Cells ou Terraria, avec une ambiance sombre adaptée à l'environnement souterrain du jeu. Le choix d'un pixel art volontairement contrasté — silhouettes nettes sur fonds sombres — répond directement à la contrainte du jeu : dans un monde obscur, les éléments révélés par l'écho doivent être immédiatement lisibles. Les palettes de couleurs ont été définies en tenant compte de ce contexte d'affichage particulier.

Deux personnages jouables ont été entièrement créés et intégrés, avec toutes leurs animations : idle, course, saut, prise de dégâts, deux attaques et mort. La production de ces animations a été significativement plus longue que prévu : chaque animation requiert entre 4 et 12 frames, et chaque frame doit être dessinée à la main dans Aseprite. La multiplication des animations par le nombre de personnages et d'ennemis représente des centaines de frames individuelles.

Quatre ennemis ont également été conçus, animés et intégrés, chacun avec ses propres animations adaptées à son archétype (patrouilleur, garde, gardien, boss). Enfin, deux types de potions ont été dessinés et intégrés comme drops potentiels des ennemis, complétant la palette d'objets visuels présents sur la carte. La feuille de sprites complète des personnages jouables et des ennemis, telle qu'exportée depuis Aseprite, est visible en annexe B (section B.3). La map finale telle qu'elle apparaît dans Tiled Map Editor et dans le jeu est présentée dans les sections B.2 du même chapitre, permettant de mesurer visuellement l'ampleur du travail de level design accompli.

4.3.5 Joies et peines

- Voir un personnage qu'on a dessiné frame par frame se mettre à courir ou attaquer dans le jeu. Chaque animation intégrée donne l'impression que le jeu gagne en vie et en cohérence. Le moment où le sprite de course a fonctionné pour la première fois dans Pygame, avec le rythme des frames correctement cadencé, est un souvenir fort du projet.
- Après tous les problèmes d'intégration, voir la map fonctionner correctement sur l'intégralité du niveau. L'aide spontanée de Jules est quelque chose que j'ai beaucoup apprécié ; elle illustre bien l'esprit d'entraide qui a animé l'équipe dans les moments difficiles.
- La cohérence visuelle finale du jeu. Voir les sprites, la map et les effets d'écho coexister dans une même atmosphère sombre et cohérente a été une belle récompense pour tous les efforts de direction artistique.

- L’animation pixel art prend beaucoup plus de temps qu’on ne le pense au départ. Quand on multiplie le nombre d’animations par le nombre de personnages et d’ennemis, ça représente des dizaines de frames à dessiner et peaufiner. Cette sous-estimation initiale a été l’une des principales sources de pression en fin de projet.
- Les problèmes de calques sur la map ont été frustrants. La cause n’était pas évidente à identifier au départ et le débogage a pris du temps qui aurait pu être utilisé ailleurs. Rétrospectivement, définir dès le départ une convention stricte de nommage et d’usage des calques aurait évité la majeure partie de ces problèmes.
- L’impossibilité de finaliser les animations du troisième personnage jouable dans les délais. Ce personnage est présent dans le jeu mais avec des animations incomplètes, ce qui crée une légère asymétrie visuelle en mode multijoueur.

4.3.6 Lien avec le travail des autres membres

La map est le terrain sur lequel tout le reste prend vie. Elle interagit directement avec la physique de Gaspard pour les collisions, avec l’IA d’Amaury pour le placement des ennemis et les règles de patrouille, et avec le réseau de Jules pour la synchronisation de la carte de visibilité entre joueurs. Les sprites que j’ai produits sont chargés et animés par les systèmes développés par Amaury et Gaspard. La direction artistique globale a également influencé les choix d’interface de Florian, qui a repris la palette de couleurs sombres et les effets néon pour les menus.

4.4 Gaspard Sapin

Mécaniques de jeu — Déplacements — Boss

4.4.1 Périmètre de la mission

Dans le projet Echo, je me suis occupé de toutes les mécaniques fondamentales du gameplay : la physique du personnage joueur (mouvement, gravité, collisions), les capacités spéciales (double saut, dash), le système d’écho (radial

et directionnel), et le boss principal du jeu. Ces éléments constituent la colonne vertébrale du ressenti du joueur ; bien les concevoir était essentiel pour que le jeu soit agréable à prendre en main.

La responsabilité du gameplay me plaçait dans une position critique : si la physique du joueur était mauvaise, tout le reste du jeu en pâtissait. Chaque décision sur les constantes de mouvement (vitesse, gravité, hauteur de saut) avait des répercussions directes sur le level design d'Eric et sur l'équilibrage général du jeu. Cette position centrale m'a conduit à beaucoup itérer et à maintenir une communication étroite avec les autres membres.

J'avais également assuré le rôle de chef de projet lors du lancement du projet, avant de le céder à Amaury en phase 2 afin de me concentrer pleinement sur les mécaniques de jeu.

4.4.2 Évolution du rôle au fil des phases

Lors de la première soutenance, le système de déplacement devait être finalisé à 100 % et la mécanique d'écho à environ 60 %. Le dash et le double saut étaient prévus pour la phase 2, et le boss pour la phase 3.

En phase 1, le déplacement au sol, le saut, la gravité et la détection des collisions ont été implémentés, atteignant 85 % de l'objectif. La mécanique d'écho radial a été entièrement implémentée, dépassant les prévisions à 60 %. Le principal problème rencontré en phase 1 concernait les collisions avec les angles de tuiles, résolues en optant pour une approche AABB sur axes séparés.

En phase 2, le double saut et le dash ont été intégrés, complétant le système de déplacement à 100 %. L'écho directionnel a également été ajouté. Une attention particulière a été portée à l'équilibrage : le dash trop rapide permettait de traverser des zones entières ; plusieurs itérations ont été nécessaires pour trouver les bonnes valeurs. En parallèle, les premières réflexions sur le boss ont conduit à choisir le Demon Slime comme adversaire final, et à définir son architecture en FSM.

En phase 3, le boss a été entièrement développé et intégré, les assistances de game feel (coyote time, jump buffer) ont été ajoutées, et l'ensemble des constantes a été rééquilibré à la lumière des tests finaux.

4.4.3 Choix techniques

- **Physique AABB à pas fixe** : Le mouvement du joueur est calculé séparément sur les axes X et Y, avec résolution des collisions indépendante sur chaque axe. Cette approche évite les artefacts de coin (le personnage qui se coince sur une arête de tuile lors d'un saut rasant) et garantit une physique prévisible. Elle est plus simple à implémenter que les approches continues et suffisante pour un jeu de plateforme 2D.
- **Machine à états finis pour le boss** : La classe `DemonSlimeBoss` implémente une FSM avec cinq états (IDLE, WALK, CLEAVE, TAKE_HIT, DEATH). Chaque transition est explicite et contrôlée par des timers et des distances. Ce choix rend le comportement du boss lisible et débogable : il est toujours possible de savoir dans quel état se trouve le boss et pourquoi.
- **Algorithme DDA pour l'écholocalisation** : L'algorithme Digital Differential Analyzer permet de tracer des rayons de manière efficace sur une grille de tuiles. Il est utilisé à la fois pour l'écho radial (360 rayons) et l'écho directionnel (cône $\pm 25^\circ$). Le DDA est préféré à un raycasting naïf pixel-par-pixel car il saute directement de frontière de tuile en frontière de tuile, réduisant considérablement le nombre d'itérations.
- **Précalcul des directions** : Les 360 vecteurs directionnels de l'écho sont précalculés à l'initialisation de la carte, évitant les appels à `math.cos` et `math.sin` à chaque frame. Ce type d'optimisation, invisible à l'exécution, est déterminant pour maintenir le framerate lors de l'activation de l'écho.
- **Assistances de game feel** : Le coyote time (100 ms de grâce après avoir quitté un bord) et le jump buffer (mémorisation d'un appui Espace anticipé de 100 ms) ont été ajoutés en phase 3. Ces mécanismes, invisibles pour le joueur, rendent les contrôles nettement plus réactifs et agréables, en compensant les imprécisions naturelles de la saisie humaine.

4.4.4 Réalisations concrètes

Réalisations concrètes

Physique du joueur :

Le système de déplacement gère : la marche (5 px/frame), le saut (force 13), la gravité (0.6 u/frame), le double saut (force 10, une seule utilisation par saut), et le dash (128 px en 150 ms avec cooldown de 600 ms, limité à une utilisation en l'air). L'équilibrage de ces constantes a nécessité de nombreuses sessions de test : le saut initial était trop haut pour la hauteur des plateformes d'Eric, et le dash trop rapide permettait de traverser des sections entières sans les explorer.

Des assistances de game feel ont été ajoutées en phase 3 : un coyote time de 100 ms (on peut encore sauter brièvement après avoir quitté le sol), un jump buffer de 100 ms (un appui Espace anticipé est mémorisé), et une coupe de saut variable selon la durée d'appui sur la touche. Ces ajouts, réalisés tardivement mais décisifs, transforment un déplacement fonctionnel en un déplacement plaisant.

Boss Demon Slime :

Le boss est le combat final du jeu. Son concept initial (le Capitaine Vex, un boss invisible se révélant à l'écho) a été remplacé par le Demon Slime, plus adapté aux contraintes de temps et à l'architecture de sprites disponibles. Sa FSM gère un rayon de détection (300 px), un rayon d'attaque basé sur la taille réelle du sprite, un cooldown d'attaque (1200 ms) et des animations synchronisées avec l'état. La hitbox de l'attaque (cleave) n'est active que sur un sous-ensemble de frames précis, garantissant un combat équitable. Le développement du boss a constitué l'une des phases les plus intenses du projet, concentrée sur les six dernières semaines de développement. La logique de décision de la FSM, notamment la fonction `_decide_next_state` qui arbitre les transitions entre états en fonction de la distance au joueur et du cooldown d'attaque, est reproduite et commentée en annexe C (section C.3).

Écholocalisation :

L'écho radial lance 360 rayons via DDA depuis la position du joueur. La révélation est progressive (sur 600 ms), simulant la propagation d'une

onde sonore. Ce délai de révélation, absent dans le premier prototype (révélation instantanée), a été ajouté après les premiers tests qui ont montré que la révélation instantanée brisait l’immersion. L’écho directionnel projette le même nombre de rayons dans un cône de $\pm 25^\circ$, avec une portée double (300 px), offrant une exploration ciblée à longue portée. L’implémentation complète de l’algorithme DDA, avec la gestion des pas sur les axes X et Y et la révélation des tuiles voisines, est détaillée en annexe C (section C.2). Une capture d’écran illustrant l’écho actif sur la map définitive est également disponible en annexe B (section B.4).

4.4.5 Joies et peines

- Voir le premier écho révéler correctement les contours de la map en temps réel. Voir l’algorithme DDA fonctionner pour la première fois a été un moment fort du projet, d’autant que le passage du raycasting naïf au DDA avait demandé plusieurs jours de recherche et d’implémentation.
- Quand le boss a attaqué pour la première fois et tué le personnage de test, on a su que ça allait vraiment faire son effet en jeu. Ce moment a validé des semaines de travail sur la FSM et les hitboxes.
- L’ajout du coyote time et du jump buffer en phase 3, qui ont transformé le ressenti des déplacements. Ces petits détails, pourtant invisibles, changent fondamentalement l’expérience de jeu.

Peines

- L’équilibrage du dash a été frustrant : trop rapide, il permettait de traverser des zones entières sans les explorer ; trop lent, il n’ajoutait rien. Plusieurs itérations ont été nécessaires pour trouver le bon compromis, et les retours des autres membres pendant les tests ont été essentiels.
- La gestion de la gravité du boss a introduit des bugs subtils difficiles à reproduire, notamment lors des transitions entre états de la FSM. Un boss qui passe de l’état WALK à TAKE_HIT en plein saut pouvait se retrouver suspendu dans les airs pendant quelques frames, brisant

l'immersion.

- Le retard accumulé sur le boss en phase 2 (10 % au lieu de 50 %) a créé une pression importante en phase 3. Compresser en six semaines ce qui aurait dû être fait sur trois mois a conduit à quelques raccourcis que j'aurais voulu éviter.

4.4.6 Lien avec le travail des autres membres

Les mécaniques que j'ai développées sont la colonne vertébrale du jeu. La physique du joueur est exécutée par le serveur de Jules à chaque tick. L'écho interagit avec la carte d'Eric pour révéler les tuiles. Le boss est géré par la classe `BossRoom` qui intègre les collisions avec la map et les attaques des joueurs. Le coyote time et le jump buffer ont été des ajouts tardifs mais décisifs pour que le jeu soit agréable à prendre en main. Les constantes de déplacement (vitesse, hauteur de saut) ont été finement calibrées en collaboration avec Eric pour qu'elles soient cohérentes avec la géométrie de sa map.

4.5 Jules Cohen

Réseau et multijoueur — Lore — Jouabilité

4.5.1 Périmètre de la mission

Dans le projet Echo, je me suis occupé de l'architecture réseau du jeu (serveur autoritaire, protocole TCP, relay WAN), du système de sauvegarde, de l'intégration du lore via les pancartes interactives, et de la jouabilité générale. C'est un domaine transversal : le réseau touche à toutes les autres parties du jeu, ce qui m'a amené à collaborer étroitement avec chaque membre de l'équipe.

La nature de ma mission impliquait d'être au courant de l'état des autres modules en permanence : pour construire un serveur autoritaire, il faut comprendre exactement ce que chaque entité attend comme données d'entrée et produit comme données de sortie. Ce rôle d'intégrateur m'a parfois placé en position de médiation technique entre les membres, servant de lien entre des contributions développées en parallèle.

4.5.2 Évolution du rôle au fil des phases

Lors de la première soutenance, le réseau devait atteindre environ 30 % d'avancement, avec une communication basique entre instances du jeu. La sauvegarde et le lore étaient prévus pour les phases suivantes.

En phase 1, un premier prototype de communication réseau a été mis en place. Le protocole initial reposait sur des échanges JSON, mais s'est révélé trop lent pour un jeu en temps réel. Il a été remplacé par un système pickle compressé avec préfixe de longueur, nettement plus adapté. La communication fonctionnait en local, avec modification manuelle du pare-feu nécessaire.

En phase 2, le système multijoueur a été rendu fonctionnel en réseau local, puis via Hamachi pour le jeu en ligne. C'est également au cours de cette phase que j'ai apporté mon aide à Eric sur les problèmes de calques Tiled, une initiative spontanée qui a permis de débloquer l'intégration de la map. La sauvegarde JSON sur 3 slots a été implémentée.

En phase 3, le relay TCP a été développé — permettant le jeu en WAN sans Hamachi ni redirection de port — et le système de lore (pancartes interactives) a été intégré en collaboration avec Amaury.

4.5.3 Choix techniques

Choix techniques

- **Architecture client-serveur autoritaire** : Le serveur possède toute la logique de jeu (physique, IA, combats). Les clients n'envoient que leurs inputs et reçoivent l'état complet du monde. Ce modèle garantit l'absence de triche et une synchronisation parfaite entre tous les joueurs. Il implique cependant que le serveur soit exécuté par l'hôte, ce qui est acceptable pour un jeu coopératif entre amis.
- **Protocole TCP avec préfixe de longueur** : Les paquets sont sérialisés avec `pickle` et précédés de 4 octets indiquant leur taille, permettant la reconstruction fiable des messages même en cas de fragmentation TCP. Ce choix, appris lors du passage du JSON au pickle en phase 1, est devenu la base robuste sur laquelle tout le reste du système réseau a été construit.

- **Serveur relay TCP pour le jeu en ligne** : Un serveur relay standalone permet à deux joueurs de se connecter via un code de salle à 6 caractères, sans redirection de port. Ce composant, développé en phase 3, n'était pas prévu dans le CdST initial et constitue l'une des contributions les plus valorisantes du projet.
- **Delta de visibilité pour optimiser la bande passante** : Plutôt que d'envoyer la carte de visibilité entière à chaque frame, seules les tuiles nouvellement révélées sont envoyées, réduisant drastiquement la charge réseau. Ce mécanisme, mis au point après que les premiers tests avec plusieurs joueurs aient révélé des problèmes de bande passante, a nécessité une réflexion approfondie sur la gestion des états différentiels.

4.5.4 Réalisations concrètes

Réalisations concrètes

Serveur autoritaire :

Le serveur tourne dans un thread dédié et gère jusqu'à 3 connexions simultanées. Il exécute la boucle de jeu à 60 Hz (physique, IA, combats, collectes) et diffuse l'état du monde à tous les clients à 10 Hz. Chaque client dispose de son propre thread de réception et d'envoi pour éviter les blocages. Cette architecture multi-threadée a nécessité une attention particulière aux conditions de course, notamment pour les collectes d'objets simultanées signalées par Amaury.

Le débogage du serveur a été une tâche particulièrement chronophage : les bugs réseau sont souvent non-déterministes et difficiles à reproduire en dehors de conditions réelles de latence. Plusieurs sessions de test sur différents réseaux ont été nécessaires pour valider la robustesse du système face aux déconnexions inattendues et aux paquets retardés. Le protocole TCP à préfixe de longueur qui sous-tend tout ce système — les fonctions `recv_complet` et `send_complet` qui garantissent la reconstruction fiable des paquets fragmentés — est intégralement reproduit et commenté en annexe C (section C.4). L'architecture globale des modules réseau est par ailleurs décrite en annexe A (section A.2).

Sauvegarde :

Le module de sauvegarde gère 3 slots JSON stockant le checkpoint, la carte de visibilité, l'argent et les améliorations débloquentes. La sauvegarde est automatique au contact des tuiles checkpoint et fonctionne exclusivement côté hôte. Le format JSON a été choisi pour sa lisibilité et sa facilité de débogage, même si la taille des fichiers est plus importante qu'un format binaire.

Relay WAN :

Le serveur relay permet à deux joueurs sur des réseaux distincts de se connecter via un code à 6 lettres, sans ouvrir de port sur leur routeur. Le code est généré aléatoirement à chaque hébergement et expire après 5 minutes d'inactivité. Ce composant est déployable sur n'importe quel VPS et constitue la solution la plus accessible pour jouer en ligne. Sa réalisation a nécessité de comprendre le fonctionnement du NAT et des relais TCP, concepts qui n'avaient pas encore été abordés en cours.

Aide au level design :

Constatant qu'Eric était bloqué sur des problèmes de calques Tiled en fin de phase 2, j'ai proposé de l'aider sur la correction de la map. Cette initiative spontanée a conduit à établir une convention documentée d'organisation des calques, qui a ensuite servi de référence pour la suite du développement. Au-delà de l'aspect technique, c'est un moment qui illustre bien l'esprit d'équipe qui a animé le projet dans ses moments difficiles.

4.5.5 Joies et peines

Joies

- Le moment où le relay a fonctionné pour la première fois : voir deux machines distantes jouer ensemble sans aucune configuration réseau grâce au code de salle. C'est le genre de fonctionnalité qui fait vraiment la différence dans l'expérience utilisateur, et dont la réalisation technique sous-jacente (gestion du NAT, tunneling TCP) représentait un défi réel.
- Avoir aidé Eric sur la map Tiled alors qu'il était bloqué. Au-delà de la technique, c'est un moment qui illustre bien l'esprit d'équipe du

projet et qui a eu un impact concret sur l'avancement global.

- La première session de jeu à trois joueurs sur le relay, depuis trois réseaux différents. Voir le serveur tenir la charge sans planter a été une validation de toutes les heures passées à le rendre robuste.

- La synchronisation de la carte de visibilité entre plusieurs joueurs a été un problème délicat. Chaque joueur possède sa propre `vis_map`, mais le serveur doit les maintenir à jour sans envoyer des mégaoctets à chaque frame. Le système de delta a résolu le problème, mais sa mise au point a pris beaucoup de temps et plusieurs approches intermédiaires ont dû être abandonnées.
- La gestion des déconnexions inattendues (crash d'un client, perte réseau) a nécessité un travail important de robustesse pour que le serveur continue de fonctionner sans planter. Chaque cas limite — déconnexion en plein combat, client qui freeze, paquets dans le désordre — a dû être explicitement traité.
- Le changement de 60 FPS à 30 FPS en phase 2 pour améliorer la stabilité réseau a été une décision difficile à accepter. Réduire le framerate pour des raisons de synchronisation réseau est un compromis peu valorisant, même s'il était techniquement justifié.

4.5.6 Lien avec le travail des autres membres

Le serveur que j'ai développé est le cœur qui fait fonctionner ensemble toutes les contributions de l'équipe : il exécute la physique de Gaspard, les collisions sur la map d'Eric, les objets d'Amaury, et envoie le tout aux clients de Florian. Cette interdépendance totale de ma contribution avec toutes les autres parties du projet donne une mesure concrète de ce que signifie travailler sur un composant d'intégration dans un projet en équipe.

Chapitre 5

Récit de la réalisation collective

5.1 Vue d'ensemble du déroulement

Le projet [ECHO] a démarré en septembre 2025 par une phase d'idéation intense. Plusieurs concepts ont été explorés avant de converger sur la mécanique d'écholocalisation, qui s'est immédiatement imposée comme originale et techniquement stimulante. Les premières semaines ont été consacrées à la rédaction du CdST et à la mise en place de l'environnement de développement commun.

5.1.1 Phase 1 : exploration et preuve de concept

La phase 1 (jusqu'à janvier 2026) a été celle de l'exploration. Chaque membre a travaillé sur sa partie en relative autonomie, se retrouvant régulièrement pour intégrer les briques entre elles. Le premier écho fonctionnel, le premier saut, la première connexion réseau : autant de petites victoires qui ont cimenté la cohésion de l'équipe.

Cette phase a également permis de détecter rapidement les choix d'architecture à reconsidérer. Le premier protocole réseau, basé sur des échanges JSON, s'est révélé trop lent pour un jeu en temps réel ; il a été remplacé par un système pickle compressé avec préfixe de longueur, plus adapté aux contraintes de la boucle de jeu.

5.1.2 Phase 2 : intégration et décisions difficiles

La phase 2 (janvier-mars 2026) a été la plus dense. L'intégration de toutes les parties a commencé à révéler des problèmes de dépendance : la map devait être stable pour l'IA, les animations devaient être prêtes pour les personnages. Des décisions difficiles ont dû être prises : reporter l'IA, simplifier certaines

animations, limiter temporairement les zones jouables.

Ces ajustements, bien que douloureux, ont permis de livrer une démo jouable et stable à la soutenance 2. C'est aussi durant cette phase que le relay WAN a été développé, une extension non prévue initialement mais qui a considérablement amélioré l'expérience multijoueur.

5.1.3 Phase 3 : aboutissement et finitions

La phase 3 (mars-mai 2026) a été celle de l'aboutissement. Forts de la base solide construite en phase 2, tous les éléments reportés ont pu être intégrés : IA des ennemis, animations complètes, map entière, boss, relay WAN, lore, effets sonores. La dernière semaine a été consacrée au packaging, aux tests finaux et à la rédaction du rapport.

5.2 Les joies collectives

- **La première session multijoueur fonctionnelle** : voir deux personnages se déplacer simultanément sur la même map, synchronisés en temps réel, a été un moment d'euphorie collective que personne n'oubliera.
- **Le premier combat complet contre le boss** : le Demon Slime qui attaque, prend des dégâts, passe par ses différentes animations et meurt — un moment qui a validé des mois de travail.
- **La carte complète dévoilée par l'écho** : voir pour la première fois l'intégralité des tuiles se révéler progressivement sous les ondes des joueurs a donné une mesure concrète de l'ampleur de ce qui avait été accompli.
- **Le relay WAN** : jouer en ligne depuis deux réseaux différents avec un simple code à six lettres, sans aucune configuration réseau.

5.3 Les peines collectives

Peines

- **Le retard de l'IA en phase 2** : avoir dû annoncer à la soutenance 2 que l'intelligence artificielle n'était pas encore intégrée au livrable officiel a mis en évidence l'importance de la gestion des dépendances entre modules.
- **Les conflits Git** : plusieurs fusions de branches difficiles, notamment

lors de l'intégration simultanée du réseau et des mécaniques de jeu, ont nécessité des sessions de débogage longues et stressantes.

- **L'équilibrage du jeu** : trouver les bons paramètres (vitesse, force des sauts, durée des cooldowns, difficulté des ennemis) a demandé de nombreuses itérations et des heures de tests.
- **La cohérence réseau sous charge** : les premiers tests avec trois joueurs simultanés ont révélé des problèmes de synchronisation qui ont demandé une refonte partielle du système d'envoi des données.

5.4 Ce que le groupe a appris

Cette année de projet a permis des apprentissages considérables, aussi bien sur le plan technique que sur le plan humain.

Sur le **plan technique**, nous avons découvert la réalité du développement d'un jeu vidéo complet : la gestion de la physique, la complexité de la synchronisation réseau, l'importance de l'architecture logicielle pour la maintenabilité, et les subtilités de l'algorithmique appliquée (DDA, FSM). Nous avons également appris à utiliser des outils professionnels (Tiled, Aseprite, PyInstaller) et à intégrer leurs formats dans notre propre code.

Sur le **plan organisationnel**, nous avons compris que la gestion des dépendances entre modules est aussi importante que le code lui-même. Anticiper ce que chaque partie du projet a besoin pour fonctionner, et planifier les livraisons en conséquence, est une compétence que nous n'aurions pas pu acquérir autrement.

Sur le **plan humain**, nous avons appris à travailler en équipe malgré des emplois du temps différents, à faire des compromis, à communiquer clairement sur les blocages, et à faire confiance aux autres membres pour livrer leur partie.

5.5 Ce qui serait fait différemment

Avec le recul, plusieurs décisions auraient pu être prises différemment :

- **Créer une map de test minimaliste dès le début** : une petite scène de test aurait permis de développer et d'intégrer l'IA bien plus tôt, sans attendre que la map finale soit stable.

- **Définir des interfaces entre modules dès la phase 1** : documenter formellement les contrats entre les modules (format d'état réseau, méthodes sur les entités) aurait réduit les conflits de merge.
- **Tester en multijoueur plus régulièrement** : les problèmes de synchronisation sont apparus tardivement parce que les tests étaient surtout effectués en solo.
- **Investir plus tôt dans le packaging** : la création de l'exécutable PyInstaller a révélé en fin de projet des problèmes de chemins et de ressources manquantes qui auraient été plus faciles à corriger en cours de développement.

Ces enseignements se retrouvent en filigrane dans l'organisation finale du code, telle que documentée en annexe A : la séparation stricte entre `core/`, `reseau/`, `ui/` et `utils/` répond directement aux problèmes de couplage identifiés en cours de projet. Cette architecture, si elle avait été formalisée dès la phase 1 plutôt qu'affinée progressivement, aurait sans doute évité plusieurs des conflits de merge décrits dans la section précédente.

Chapitre 6

Présentation du résultat final

6.1 Description fonctionnelle du livrable

Le livrable final d'[ECHO] est un jeu de plateforme 2D coopératif complet, packagé sous forme d'exécutable Windows autonome.

6.1.1 Gameplay central

Le joueur évolue dans une map entièrement plongée dans l'obscurité. En appuyant sur E, il émet un écho radial (360°, 150 px) qui révèle définitivement les tuiles proches. La révélation est progressive, simulant la propagation d'une onde sonore sur 600 ms. La capacité d'écho directionnel (cône $\pm 25^\circ$, 300 px) est débloquable.

6.1.2 Déplacements et capacités

Le personnage peut marcher, sauter, effectuer un double saut et un dash. Ces trois capacités avancées sont débloquables via des orbes sur la carte et sauvegardées entre les sessions. Le double saut permet d'atteindre des zones en hauteur, le dash permet de franchir des obstacles ou d'esquiver des attaques.

6.1.3 Combat et progression

Le joueur attaque au corps à corps (portée 40 px). Trois types d'ennemis patrouillent les couloirs : le patrouilleur (1 PV, rapide), le garde (2 PV, standard) et le gardien (3 PV, lent mais résistant). Un boss final (Demon Slime) attend le joueur dans sa salle dédiée, avec une machine à états complète et des hitboxes d'attaque précises.

6.1.4 Système d'âmes

Les ennemis lâchent des âmes à leur mort. À sa propre mort, le joueur laisse une âme contenant tout son argent à récupérer. Des âmes libres sont disséminées sur la map pour récompenser l'exploration. Ce système de pression, inspiré de *Dark Souls*, encourage le joueur à prendre des risques calculés.

6.1.5 Multijoueur coopératif

Jusqu'à 3 joueurs peuvent jouer ensemble, en LAN (IP directe) ou en WAN (code de salle à 6 caractères via relay TCP). Le serveur autoritaire garantit la synchronisation parfaite des états, et chaque joueur dispose de sa propre carte de visibilité.

6.1.6 Interface et accessibilité

Le jeu propose des menus complets (principal, paramètres, pause, slots de sauvegarde), un tutoriel au premier lancement, une sauvegarde automatique aux checkpoints (3 slots), une reconfiguration complète des touches, et un support bilingue français/anglais.

6.2 Comparaison avec les objectifs initiaux

Objectif initial	Réalisation	Statut
Écholocalisation radiale	Implémentée (DDA, 360 rayons, révélation progressive)	Atteint
Écho directionnel (débloquable)	Implémenté (cône $\pm 25^\circ$, 300 px)	Atteint
Déplacements complets	Implémentés et équilibrés	Atteint
Map complète avec textures	Map complète, tileset graphique définitif	Atteint
Sprites animés (joueurs)	2 personnages avec animations complètes	Atteint
IA ennemis	3 types d'ennemis avec patrouille et détection	Atteint
Pathfinding A*	Patrouille basique (A* non intégré au livrable)	Partiel
Boss complet avec FSM	Demon Slime avec 5 états et hitboxes précises	Atteint
Multijoueur coopératif réseau	LAN + relay WAN (3 joueurs max)	Atteint
Système de sauvegarde	3 slots JSON, sauvegarde auto aux checkpoints	Atteint
Audio complet	Musique + 15+ effets sonores + spatialisations	Atteint
Lore intégré	Pancartes interactives débloquables	Atteint
Site web officiel	Site ₄₇ avec roadmap, admin, bot Discord	Atteint
Packaging Windows	Exécutables PyInstall	Atteint

Les captures d'écran présentées en annexe B permettent de visualiser des éléments listés dans ce tableau : la map dans Tiled (section B.2), les sprites des personnages (section B.3) et une capture en jeu avec l'écho actif (section B.4). Ces visuels complètent utilement la description textuelle qui suit.

6.3 Démonstration écrite du déroulement d'une partie

Au lancement, un splash screen affiche le logo du jeu avec une animation de fondu, avant d'arriver sur le menu principal au design néon animé. Le joueur peut créer une nouvelle partie (choix du slot de sauvegarde), rejoindre une partie existante (IP directe ou code de salle), ou configurer ses paramètres.

En jeu, le personnage apparaît dans l'obscurité complète. Dès le premier appui sur E, une onde lumineuse se propage en cercle et révèle les contours des murs et du sol proches. Le halo autour du personnage permet une vision courte distance constante. La torche, accessible via L, illumine une zone plus grande mais fixe.

En progressant, le joueur rencontre des ennemis qui patrouillent. Une attaque corps à corps (K) leur inflige des dégâts, et leur mort libère des âmes loot qui jaillissent physiquement avec des rebonds. Les orbes colorés débloquent de nouvelles capacités. La clé dorée, gardée par le boss, ouvre la porte vers la zone finale.

Le boss est le point culminant de la progression. Il se déplace, attaque et prend des dégâts avec des animations et des hitboxes précises. Le vaincre libère la clé qui ouvre la porte finale, déclenchant une cinématique de fin.

6.4 Limites connues et pistes d'amélioration

- **Pathfinding A* non intégré** : les ennemis suivent une logique de patrouille linéaire mais ne peuvent pas naviguer intelligemment vers le joueur. L'intégration de l'algorithme A* constituerait la priorité numéro un pour une version suivante.
- **Latence réseau sur longue distance** : le relay TCP introduit une latence variable selon la qualité de la connexion.
- **Équilibrage** : le jeu a été équilibré pour des sessions de test internes ; des tests avec des joueurs externes révéleraient probablement des déséquilibres à corriger.

- **Niveau unique :** le jeu ne comporte qu'un seul niveau. L'ajout de niveaux supplémentaires ou d'une génération procédurale constituerait une évolution naturelle.

Chapitre 7

Conclusion

7.1 Bilan synthétique du projet

[ECHO] est un jeu de plateforme 2D coopératif complet, conçu et développé de A à Z en huit mois par cinq étudiants de première année. Le projet répond à l'ensemble des exigences pédagogiques : développement en Python avec Pygame, intelligence artificielle pour les ennemis, et multijoueur coopératif en réseau. Il va même au-delà sur plusieurs points, avec un serveur relay TCP original permettant le jeu en ligne sans configuration réseau, et un système d'écholocalisation par raycasting unique.

Le résultat final est un jeu jouable, stable, avec une identité artistique cohérente, une ambiance sonore complète, et une expérience multijoueur fonctionnelle aussi bien en LAN qu'en WAN. La seule limite subsistante notable (pathfinding A* non intégré au produit final) ne remet pas en cause la qualité globale du livrable.

7.2 Compétences développées

Ce projet a permis à chaque membre de développer des compétences significatives :

- **Compétences techniques** : algorithmique (raycasting, FSM, gestion réseau TCP, sérialisation), architecture logicielle orientée objet, gestion de projet avec Git, développement d'interface graphique, conception de niveau, production d'assets pixel art.
- **Compétences méthodologiques** : planification sur plusieurs mois, gestion des priorités et des dépendances, rédaction de documentation technique, tests et débogage, packaging et livraison.
- **Compétences humaines** : travail en équipe, communication à distance, gestion des conflits et des retards, prise de décision collective, présentation orale et écrite devant un jury.

7.3 Ouvertures et perspectives

Au-delà du cadre scolaire, [ECHO] est un projet qui pourrait continuer à évoluer. Les bases techniques sont solides : l'architecture serveur autoritaire pourrait supporter des maps plus grandes, plus de joueurs, et de nouveaux modes de jeu. L'intégration de l'algorithme A* donnerait aux ennemis un comportement vraiment intelligent. L'ajout de niveaux supplémentaires, voire d'une génération procédurale, transformerait Echo en une expérience réellement rejouable.

Sur le plan pédagogique, ce projet nous a donné un avant-goût de ce qu'est le développement logiciel professionnel : la rigueur nécessaire, l'importance de l'architecture, la valeur du travail en équipe, et la satisfaction de livrer quelque chose que l'on est fier de montrer.

Bibliographie et références

- [1] Pygame Project, *Pygame Documentation*, disponible sur : <https://www.pygame.org/docs/>, consulté en 2025–2026.
- [2] Thorbjørn Lindeijer et al., *Tiled Map Editor*, disponible sur : <https://www.mapeditor.org/>, consulté en 2025–2026.
- [3] Aseprite, *Aseprite Documentation — JSON format*, disponible sur : <https://www.aseprite.org/docs/>, consulté en 2025–2026.
- [4] Amit Patel, *Introduction to A* Pathfinding*, Red Blob Games, disponible sur : <https://www.redblobgames.com/pathfinding/a-star/introduction.html>, consulté en 2025.
- [5] Lode Vandevenne, *Raycasting Tutorial — Digital Differential Analysis*, disponible sur : <https://lodev.org/cgtutor/raycasting.html>, consulté en 2025.
- [6] Python Software Foundation, *Python 3 Documentation*, disponible sur : <https://docs.python.org/3/>, consulté en 2025–2026.
- [7] NumPy Developers, *NumPy Documentation*, disponible sur : <https://numpy.org/doc/>, consulté en 2025–2026.
- [8] PyInstaller Development Team, *PyInstaller Manual*, disponible sur : <https://pyinstaller.org/en/stable/>, consulté en 2026.
- [9] Supabase, *Supabase Documentation*, disponible sur : <https://supabase.com/docs>, consulté en 2025–2026.

Annexe A

Architecture technique détaillée

A.1 Structure des modules

Le projet est organisé en plusieurs modules distincts, chacun ayant une responsabilité bien définie :

- `core/` : logique de jeu (joueur, ennemis, carte, boss, objets)
- `reseau/` : communication réseau (serveur, protocole, relay)
- `ui/` : interface graphique (menus, HUD, caméra, effets visuels)
- `sauvegarde/` : persistance (slots, paramètres, checkpoints)
- `utils/` : utilitaires partagés (cache, langue, musique, logs)
- `editeur/` : éditeur de niveaux Tiled intégré

A.2 Protocole réseau

Le protocole réseau repose sur TCP avec un préfixe de longueur de 4 octets (big-endian) suivi d'un objet Python sérialisé avec `pickle` et compressé avec `zlib`. La taille maximale d'un paquet est limitée à 10 MB pour éviter les attaques par déni de service.

- **Client vers serveur** : dictionnaire d'inputs (touches pressées, actions one-shot comme l'écho ou l'interaction avec une pancarte).
- **Serveur vers client** : état complet du monde (joueurs, ennemis, âmes, boss, clés, portes, orbes, carte de visibilité).

A.3 Système d'animation Aseprite

La classe `BossAnimator` charge un fichier JSON Aseprite et découpe les frames par tag. Elle supporte les deux formats d'export Aseprite (Hash et

Array). Chaque animation est identifiée par son nom (`d_idle`, `d_walk`, etc.) et contient une liste de paires (surface, durée en ms).

Le chargement des animations est effectué de manière paresseuse, lors du premier appel à la méthode de dessin, pour éviter de charger des images depuis le thread serveur.

A.4 Système de cache et optimisations

Le module `utils/cache.py` centralise plusieurs caches partagés :

- **Cache de flip horizontal** : les surfaces flippées sont mémorisées par identifiant de surface source.
- **Cache de rendu de texte** : les surfaces rendues par `font.render` sont mémorisées par triplet (police, texte, couleur).
- **Cache de polices** : les instances `pygame.font.Font` sont mémorisées par taille.
- **Vecteurs d'écho précalculés** : les 360 vecteurs directionnels (et les variantes directionnelles) sont calculés une seule fois au chargement du module.

La grille spatiale (`construire_grille_collision`) découpe la carte en cellules de 128 px, permettant de ne tester les collisions qu'avec les tuiles voisines d'une entité, et non avec l'intégralité de la carte.

Annexe B

Aperçus du jeu

B.1 Inspirations visuelles

Les deux images suivantes illustrent les principales références artistiques qui ont guidé la direction visuelle d'[ECHO]. L'ambiance souterraine de *Silksong* et le style pixel art contrasté de *Terraria* ont été les deux références centrales de la charte graphique du projet.



FIGURE B.1 – Référence visuelle 1 — *Terraria* : pixel art contrasté et lisibilité des sprites



FIGURE B.2 – Référence visuelle 2 — *Silksong* : environnement souterrain et exploration verticale

B.2 Cartes du jeu

Les deux captures suivantes présentent la carte du jeu telle qu'elle a été conçue à la main, et un aperçu dans le jeu.

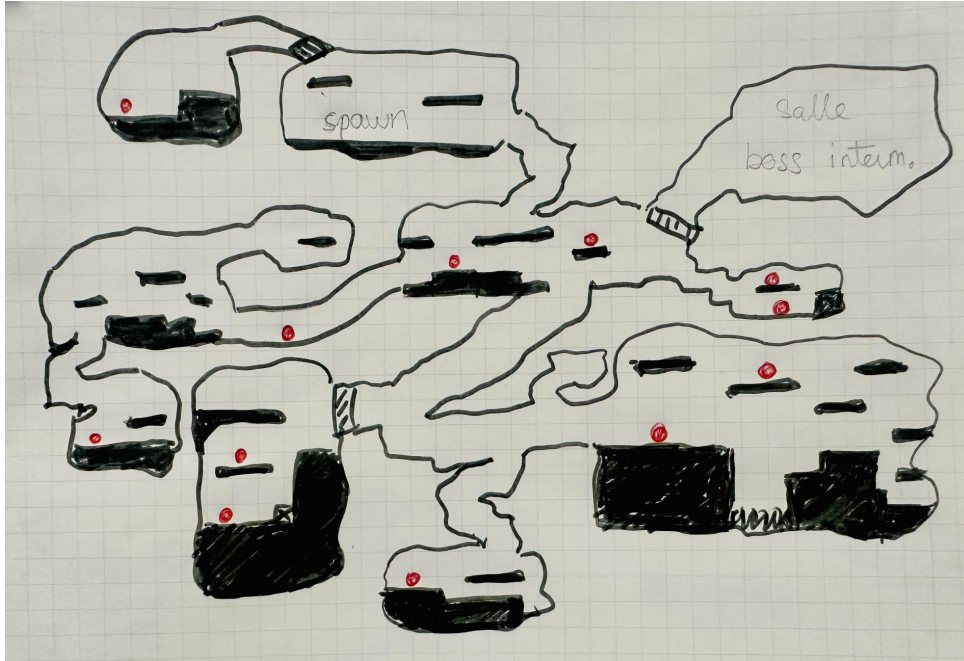


FIGURE B.3 – Vue d’ensemble de la map dans Tiled Map Editor — structure des salles et tunnels



FIGURE B.4 – Zone de jeu intégrée dans Pygame — tileset graphique définitif

B.3 Sprites des personnages



FIGURE B.5 – Feuille de sprites des personnages jouables et ennemis — réalisée avec Aseprite

B.4 Démonstration en jeu



FIGURE B.6 – Capture d'écran en jeu — écho actif révélant les contours du niveau

Annexe C

Extraits de code commentés

C.1 Algorithme de patrouille réactive des ennemis

L'intelligence artificielle des ennemis repose sur un algorithme de patrouille déterministe enrichi de règles comportementales. À chaque frame, le moteur évalue l'environnement immédiat de l'ennemi pour décider de son comportement. Le principe est simple : l'ennemi avance dans sa direction courante jusqu'à rencontrer un obstacle ou un bord de plateforme, auquel cas il fait demi-tour. Une zone de détection circulaire déclenche l'attaque lorsque le joueur s'approche.

Extrait simplifié de core/ennemi.py

```
class Ennemi:
    """
    Classe de base pour tous les ennemis du jeu.
    La configuration (vitesse, PV, taille, argent, asset) est passée
    en paramètre, permettant de définir des archétypes sans sous-classer.
    """
    def appliquer_logique(self, joueurs, carte):
        """
        Appelée à chaque tick serveur. Évalue les conditions
        de l'environnement et met à jour l'état de l'ennemi.
        """
        # Détection du sol devant l'ennemi
        prochain_x = self.rect.x + self.direction * self.vitesse
        tuile_sol = carte.get_tuile(prochain_x, self.rect.bottom + 2)
        # Détection du mur devant l'ennemi
```

```
tuile_mur = carte.get_tuile(prochain_x, self.rect.centery)

if tuile_mur in TUILES_SOLIDES or tuile_sol not in TUILES_SOLIDES:
    self.direction *= -1 # Demi-tour

# Déplacement dans la direction courante
self.rect.x += self.direction * self.vitesse

# Détection du joueur le plus proche
for joueur in joueurs:
    dist = math.hypot(
        joueur.rect.centerx - self.rect.centerx,
        joueur.rect.centery - self.rect.centery
    )
    if dist <= self.rayon_detection:
        self._attaquer(joueur)
        break

def _attaquer(self, joueur):
    """
    Inflige des dégâts si le cooldown est écoulé et que
    la hitbox d'attaque est dans les frames actives.
    """
    if self.attack_timer <= 0:
        joueur.infliger_degats(self.degats)
        self.attack_timer = self.COOLDOWN_ATTAQUE
```

Les trois archétypes d'ennemis (patrouilleur, garde, gardien) sont définis par un dictionnaire de configuration passé au constructeur de la classe de base. Cette approche évite la multiplication de sous-classes pour des entités qui ne diffèrent que par leurs paramètres numériques.

```
# Extrait de core/config_ennemis.py
CONFIGS_ENNEMIS = {
    "patrouilleur": {
        "vitesse": 3, "pv": 1, "degats": 1,
```

```

        "rayon_detection": 120, "argent": 5,
        "asset_prefix": "patrol"
    },
    "garde": {
        "vitesse": 2, "pv": 2, "degats": 1,
        "rayon_detection": 150, "argent": 10,
        "asset_prefix": "guard"
    },
    "gardien": {
        "vitesse": 1, "pv": 3, "degats": 2,
        "rayon_detection": 100, "argent": 20,
        "asset_prefix": "guardian"
    },
}

```

C.2 Algorithme DDA pour l'écholocalisation

L'écholocalisation est au cœur du gameplay d'[ECHO]. L'algorithme DDA (Digital Differential Analyzer) permet de tracer des rayons de manière efficace sur la grille de tuiles. Le principe est de calculer, pour chaque rayon, la distance nécessaire pour traverser une tuile en X et en Y, puis d'avancer pas à pas dans la direction du rayon en choisissant à chaque étape l'axe qui nécessite le moins de chemin.

```

# Extrait simplifié de core/carte.py
for cos_a, sin_a in self._directions:
    step_x = 1 if cos_a > 0 else -1
    t_delta_x = abs(TAILLE_TUILE / cos_a) if cos_a != 0 else float('inf')
    # ...
    for _ in range(portee_tuiles):
        tuile_type = self.map_data[tuile_actuelle_y][tuile_actuelle_x]
        if tuile_type in [1, 3]:
            self._reveler_voisins(...)
            break
        vis_map[tuile_actuelle_y][tuile_actuelle_x] = True
        if t_max_x < t_max_y:

```

```
t_max_x += t_delta_x
tuile_actuelle_x += step_x
else:
    t_max_y += t_delta_y
    tuile_actuelle_y += step_y
```

C.3 Machine à états du boss

La FSM du boss implémente cinq états distincts et gère les transitions de manière explicite. À chaque frame, `update()` détermine si une transition est nécessaire en fonction de la distance au joueur, du cooldown d'attaque et de l'état de l'animation courante.

```
# Extrait simplifié de core/demon_slime_boss.py
def _decide_next_state(self, distance):
    portee_reelle = int(self.sprite_w * 0.4)
    if distance <= portee_reelle and self.attack_cooldown_timer <= 0:
        self._change_state(BossState.CLEAVE)
    elif distance <= self.DETECT_RADIUS:
        if distance > portee_reelle:
            if self.state != BossState.WALK:
                self._change_state(BossState.WALK)
        else:
            if self.state != BossState.IDLE:
                self._change_state(BossState.IDLE)
    else:
        if self.state != BossState.IDLE:
            self._change_state(BossState.IDLE)
```

C.4 Protocole réseau

Le protocole TCP utilise un préfixe de longueur pour délimiter les paquets, résolvant le problème de la fragmentation TCP.

```
# Extrait de reseau/protocole.py
def recv_complet(sock):
    header = recvall(sock, 4)
```

```
taille = int.from_bytes(header, 'big')
if taille > 10_000_000:
    raise ValueError(f"Paquet suspect trop grand : {taille} octets")
return pickle.loads(zlib.decompress(recvall(sock, taille)))

def send_complet(sock, obj):
    data = zlib.compress(pickle.dumps(obj), 1)
    sock.sendall(len(data).to_bytes(4, 'big') + data)
```